

Ch5 运算符、表达式与语句

对应《C Primer Plus（第6版）》第 5 章 · 本章目标：30 分钟读懂 + 60 分钟练完 · 不覆盖：位运算、三目运算符

5.1 运算符：让程序"会算数"

上一章我们让程序"记住"了数据（变量），这一章让程序**算起来**。运算符就是告诉计算机"怎么算"的符号。最常用的五个：

运算符	含义	示例	结果
+	加	10 + 3	13
-	减	10 - 3	7
*	乘	10 * 3	30
/	除	10 / 3	3（int 除法，见 5.2）
%	求模（取余数）	10 % 3	1

```
/* ch5_01_basic.c - 加减乘除（int）*/
#include <stdio.h>

int main(void)
{
    int a = 10, b = 3;
    printf("%d + %d = %d\n", a, b, a + b);
    printf("%d - %d = %d\n", a, b, a - b);
    printf("%d * %d = %d\n", a, b, a * b);
    printf("%d / %d = %d\n", a, b, a / b);
    return 0;
}
```

编译：gcc -Wall -Wextra -o ch5_01_basic.exe ch5_01_basic.c 运行：.\ch5_01_basic.exe

运行输出（实测）：10 + 3 = 13 10 - 3 = 7 10 * 3 = 30 10 / 3 = 3

5.2 整数除法会"丢尾巴"：/ 和 % 是一对

两个 `int` 相除，结果**还是 `int`**——小数部分直接丢掉（不是四舍五入，是**截断**）。 $10 / 3 = 3$ ，剩下的那 1 就是余数，用 `%` 取出来。

口诀： / 取商，% 取余。判断“能不能整除”就看 `n % m == 0`。想保留小数？让**其中一个数是 `double`**（见 5.7）。

```
/* ch5_02_divmod.c - 整数除法 + 求模 */
#include <stdio.h>

int main(void)
{
    int minutes = 125;
    printf("125 分钟 = %d 小时 %d 分钟\n", minutes / 60, minutes % 60);

    int n = 17;
    if (n % 2 == 0) printf("%d 是偶数\n", n);
    else           printf("%d 是奇数\n", n);
    return 0;
}
```

运行输出（实测）：125 分钟 = 2 小时 5 分钟 17 是奇数

5.3 自增 ++ 自减 --：写代码偷懒的常用手段

`n++` 等价于 `n = n + 1`，`n--` 等价于 `n = n - 1`。它们还能“边用边变”：

写法	含义	例子（n 初值 5）
<code>n++</code> （后缀）	先取 <code>n</code> 的旧值用，用完再加 1	<code>x = n++</code> ; $\rightarrow x=5, n=6$
<code>++n</code> （前缀）	先加 1，再取新值用	<code>x = ++n</code> ; $\rightarrow n=6, x=6$

新手坑：一个语句里别对同一个变量又 `++` 又用。`printf("%d %d\n", n, n++)`；这种“边打印边改”的结果在 C 里**不确定**（不同编译器结果可能不同）。规矩：`++/--` 单独成行用，别混进别的表达式里。

```
/* ch5_03_incdec.c - 前缀/后缀的区别 */
#include <stdio.h>

int main(void)
{
    int a = 5;
    int x = a++;      /* x 先拿 a 的旧值 5，然后 a 变成 6 */
    int y = ++a;      /* a 先变成 7，然后 y 拿新值 7 */
    printf("a=%d x=%d y=%d\n", a, x, y);
}
```

```
    return 0;
}
```

运行输出（实测）：a=7 x=5 y=7

5.4 赋值 = 与复合赋值：+= -= *= /=

= 是把右边的值"放进"左边的盒子（变量）。注意：**左边必须是变量**，不能是 `a + b` 这种表达式。

想"把变量自己加 5"，可以写 `score = score + 5;`，更短的写法是 `score += 5;`——这叫**复合赋值**：

```
/* ch5_04_compound.c - 复合赋值 */
#include <stdio.h>

int main(void)
{
    int score = 80;
    score += 10;    /* score = 90 */
    score -= 5;     /* score = 85 */
    score *= 2;     /* score = 170 */
    printf("score = %d\n", score);
    return 0;
}
```

运行输出（实测）：score = 170

5.5 优先级：先乘除后加减，别靠猜

数学里"先乘除后加减"，C 也一样。拿不准的时候**加括号**，括号优先级最高。

```
/* ch5_05_preced.c - 优先级与括号 */
#include <stdio.h>

int main(void)
{
    int r = 2 + 3 * 4;    /* 先乘：3*4=12，再加 2 → 14 */
    int s = (2 + 3) * 4;  /* 括号先算：5*4 → 20 */
    printf("r=%d s=%d\n", r, s);
    return 0;
}
```

运行输出（实测）：r=14 s=20

优先级（高→低，本章够用）	说明
()	括号最高

++ --	自增自减
* / %	乘、除、求模（同级，从左往右）
+ -	加、减（同级，从左往右）
= += -= ...	赋值（最低）

一句话：只要表达式里混着 >2 种运算符，就多用括号表达你的意图——既防错，别人（和考试判卷）也看得懂。

5.6 表达式与语句：分号是关键

表达式是"能算出值的一串东西"：`2 + 3`、`a * b`、`n++` 都是表达式。**语句**是"完整的一句话"，结尾必须带 `;`。最常见的语句就是"表达式 + 分号"：

```
/* 表达式 vs 语句 */
2 + 3;          /* 合法但没用的语句：算完就扔了 */
a = b + c;      /* 赋值语句（也是表达式加分号）*/
printf("hi");   /* 函数调用语句 */
```

新手坑：多余的分号。在 `if` 后面多加一个分号会改变程序逻辑（第 6 章循环会再遇到）。判断"这里该不该有分号"：这句话是不是一个完整动作？是——加分号。

5.7 类型转换：int 和 double 混着算

整数和小数混在一起算，C 会把 `int` 自动"升级"成 `double` 再算（小的往大的转，不丢精度）。反过来 `double` 想存进 `int`，小数部分会被**截断**（这是警告级别的问题，开 `-Wall` 会提示）。

```
/* ch5_06_type.c - int/double 混合运算（练习册 ⑤ 有对应题）*/
#include <stdio.h>

int main(void)
{
    int a = 7;
    double half = a / 2.0;      /* a 是 int, 2.0 是 double → 结果是 3.5 */
    int trunc = a / 2;          /* 两个 int → 3, 小数被截断 */
    printf("half=%.1f trunc=%d\n", half, trunc);
    return 0;
}
```

运行输出（实测）：`half=3.5 trunc=3`

5.8 新手坑集中营（本章最容易错的 4 件事）

1. **整数除法丢小数**： $5 / 2 = 2$ 不是 2.5。想保留小数：让任一操作数是 double ($5 / 2.0$)。
2. **自增自减混进表达式**：`printf("%d", n++)` 结果不确定。单独成行用。
3. **赋值方向写反**：`=` 是把右边给左边。`5 = a;` 编译就报错（左边必须是变量）。
4. **拿不准优先级不加括号**：混着算的表达式都加括号，别赌记忆。

5.9 一分钟自测（答案见练习册）

写出结果：

- ① `int n = 10; printf("%d", n / 3);`
- ② `int m = 5; int r = m++; printf("%d %d", m, r);`
- ③ `double x = 7; printf("%.2f", x / 2);`

5.10 练习指引

做完本章内容后，去练习册完成 Ch5 配套练习；同时打开《C Primer Plus（第6版）》第 5 章，完成**复习题 1、2、4**（题号对应主教材）。改错题是最有价值的——先自己找，再看答案。

本讲义为自写原创讲解，仅作课堂伴读；示例代码均可用 `gcc -Wall -Wextra` 编译，每句「运行输出（实测）」均已实跑核对。原书题目请以正版教材为准。